

MODULE 4: SOFTWARE CONSTRUCTION 1.3
WEEK 4

Learning Outcomes:

After completing this course, you are expected to demonstrate the following:

1. Explain threading, disadvantages when a system performs more operation and how to secure a data when users interact to the system.

A. Engage

Trivia

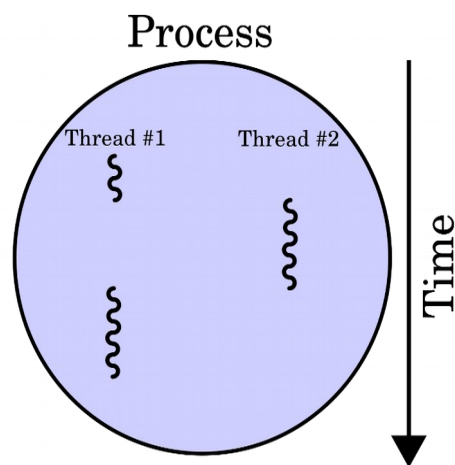


Figure 4.0

A thread is a single sequential flow of control within a program. The real excitement surrounding threads is not about a single sequential thread. Rather, it is about the use of multiple threads running at the same time and performing different tasks in a single program.

B. Explore

Video Titles:

1. Race Condition in Operating System with Example | Race Condition in OS.
2. Intermediate Java Tutorial - 26 - Learning about Threads.

YouTube Links:

1. <https://www.youtube.com/watch?v=7aF0q7NfwfA>
2. <https://www.youtube.com/watch?v=VYN-CBtPNiM>

C. Explain

What is Thread?

A thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers which hold its current working variables, and a stack which contains the execution history. A thread shares with its peer threads few information like code segment, data segment and open files. When one thread alters a code segment memory item, all other threads see that.

A thread is also called a lightweight process. Threads provide a way to improve application performance through parallelism. Threads represent a software approach to improving performance of operating system by reducing the overhead thread is equivalent to a classical process.

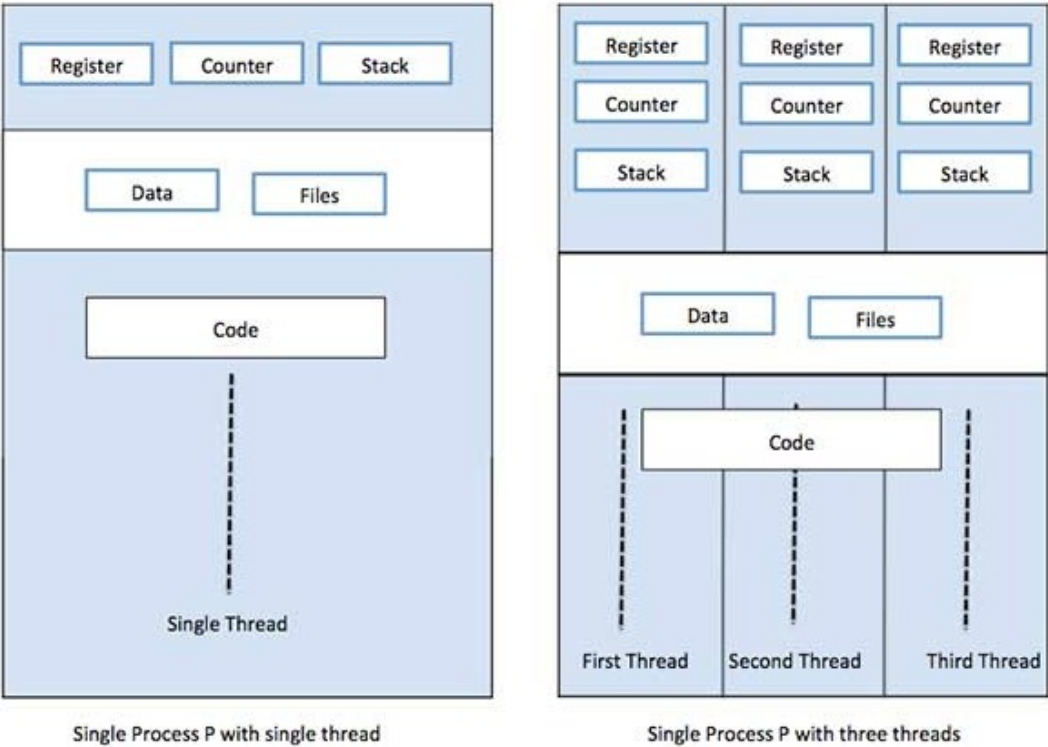


Figure 4.0

Each thread belongs to exactly one process and no thread can exist outside a process. Each thread represents a separate flow of control. Threads have been successfully used in implementing network servers and web server. They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors. The following figure shows the working of a single-threaded and a multithreaded process.

Table 4.0: The difference between Process and Thread

S.N.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

Advantages of Thread

- ✓ Threads minimize the context switching time.
- ✓ Use of threads provides concurrency within a process.
- ✓ Efficient communication.
- ✓ It is more economical to create and context switch threads.
- ✓ Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

D. Elaborate

Race Condition

A race condition is an undesirable situation that occurs when a device or system attempts to perform two or more operations at the same time, but because of the nature of the device or system, the operations must be done in the proper sequence to be done correctly.

A simple example of a race condition is a light switch. In some homes there are multiple light switches connected to a common ceiling light. When these types of circuits are used, the switch position becomes irrelevant. If the light is on, moving either switch from its current position turns the light off. Similarly, if the light is off, then moving either switch from its current position turns the light on. With that in mind, imagine what might happen if

two people tried to turn on the light using two different switches at the same time. One instruction might cancel the other or the two actions might trip the circuit breaker.

Race conditions are most associated with computer science. In computer memory or storage, a race condition may occur if commands to read and write a large amount of data are received at almost the same instant, and the machine attempts to overwrite some or all the old data while that old data is still being read. The result may be one or more of the following: a computer crash, an "illegal operation," notification and shutdown of the program, errors reading the old data or errors writing the new data. A race condition can also occur if instructions are processed in the incorrect order.

Suppose for a moment that two processes need to perform a bit flip at a specific memory location. Under normal circumstances the operation should work like this:

Table 4.1

Process 1	Process 2	Memory Value
Read value		0
Flip value		1
	Read value	1
	Flip value	0

In this example, Process 1 performs a bit flip, changing the memory value from 0 to 1. Process 2 then performs a bit flip and changes the memory value from 1 to 0. If a race condition occurred causing these two processes to overlap, the sequence could potentially look more like this:

Table 4.2

Process 1	Process 2	Memory Value
Read value		0
	Read value	0
Flip value		1
	Flip value	1

In this example, the bit has an ending value of 1 when its value should be 0. This occurs because Process 2 is unaware that Process 1 is performing a simultaneous bit flip.

Security vulnerabilities caused by race conditions.

When a program that is designed to handle tasks in a specific sequence is asked to perform two or more operations simultaneously, an attacker can take advantage of the time gap between when the service is initiated and when a security control takes effect to create a deadlock or thread block situation. With deadlock, two or more threads must wait for a lock in a circular chain. This defect can cause the entire software system to halt because such locks can never be acquired if the chain is circular. Thread block can also dramatically impact application performance. In this type of concurrency defect, one thread calls a long-running operation while holding a lock and preventing the progress of other threads.

Preventing race conditions

In computing environments, race conditions can be prevented by serialization of memory or storage access. This means if read and write commands are received close together, the read command is executed and completed first by default. In a network, a race condition may occur if two users attempt to access an available channel at the same instant, and neither computer receives notification the channel is occupied before the system grants access. Statistically, this kind of coincidence will most likely occur in networks with long lag times, such as those that use geostationary satellites. To prevent such a race condition from developing, a priority scheme must be devised. For example, the subscriber whose username begins with the earlier letter of the alphabet (or the lower numeral) may get priority by default when two subscribers attempt to access the system within a prescribed increment of time. Hackers can take advantage of race-condition vulnerabilities to gain unauthorized access to networks.

Race conditions occasionally occur in logic gates when certain inputs come into conflict. Because the gate output state takes a finite, nonzero amount of time to react to any change in input states, sensitive circuits or devices following the gate may be fooled by the state of the output, and thereby not operate properly.

Secure Coding: Input Validation and Data Sanitation

In today's security sensitive environments, it is critical to focus on secure software development methods. Quite a few 'holes' exist in software due to (a) legacy practices and (b) poor programming practices that exist in many software developments shops. One of the key factors in developing secure software is to validate (e.g., check and verify) the input. Without input validation as a primary software development approach, the implemented software could be susceptible to evil attackers.

Input validation is considered a "best practice" for coding, regardless of the software that is being used for the development of applications (Java, C++, C, and yes COBOL, etc.). With input validation, the software developer configures all input fields to only accept very specific types of data with a predefined length. Some common types of validation include verifying that proper characters are inputted, implementing boundary or range checking, blocking HTML code from being entered into an entry field, and preventing the use of certain characters that may cause erroneous problems with the application code. The program should check for the specific requirements of the program and will either (a) provide some (limited) comments about the format that is suitable for the input fields and/or (b) will sanitize the data.

Data Sanitation takes a deeper approach to input validation. Data sanitation refers to ways that input is modified by the code. In general, data sanitation 'cleans up' what the user inputs during the validation process. For example, if the user enters in double single quotes, the program can (and should) clean up these, since entry of extraneous data could lead to code that may be executed by an evil attacker.

Both input validation and data sanitation are used to increase the security of a developed application. Both (or either) should “throw an error” and provide the user with some (but limited) comments that informs the user about the input error. As an example, when programming for web applications, the lack of validation and sanitation may open the application to attacks like buffer overflow attacks, SQL injection, command injection, and cross-site scripting attacks (W3School, 2018). The risk for not validating nor sanitizing input runs the risk of opening the code, the system and the infrastructure using the code to exploitation.

E. Evaluate

ASSESSMENT:

Instructions: Write your answer on the Answer Sheet (AS) provided in this module. (50 pts)

1. Create a table that shows the 6 differences between process and thread (50 pts)

LABORATORY ACTIVITY 4.1

1. Give a sample program in java that shows how a thread works, your resources can be found at video resources folder, Week4_Software Construction1.1 name of the video is “Threads”. Goodluck!

References:

- 1. *Operating System - Multi-Threading*
 - *Link*
 - https://www.tutorialspoint.com/operating_system/os_multi_threading.htm
- 2. *Race Condition*
 - *Link*
 - <https://searchstorage.techtarget.com/definition/race-condition>
- 3. *Secure Coding: Input Validation and Data Sanitation*
 - *Link*
 - <https://highervista.medium.com/secure-coding-input-validation-and-data-sanitation-9d16fb14c94d>

Facilitated By:		
Name	:	
MS Teams Account (email)	:	
Smart Phone Number	:	